# IncidentFlow

## Build It Yourself

A complete, illustrated walkthrough of a production incident-management platform — every screen, every file, every command, and every bug that had to be fixed.

---

Written for someone who can open a project in VS Code and nothing more. No Laravel, React, Docker or SQL assumed.

---

Project directory: `D:\IncidentFlow`

# Contents

# How to read this book

This book has one goal: that you could rebuild IncidentFlow *by hand*, without help, having read it.

That is a higher bar than "understand the code". It means you need to know not only what each file contains, but why it exists, what would break without it, which command creates it, and in what order the pieces have to be built.

## What is in here

**Part I** **Understanding.** What the system does, the shape of it, and where every file lives. Read this once, slowly. Nothing later makes sense without it.

**Part II** **The guided tour.** Every screen, as a picture, next to the code that produces it and the file path it lives at. This is the bulk of the book.

**Part III** **Running it.** Every command, every connection, every port. Written for someone who has only ever opened a folder in VS Code.

**Part IV** **Building it from nothing.** The order in which the pieces have to be created, and why that order is forced.

**Part V** **Bugs.** Real failures from building this system — the symptom, the wrong guess, the evidence, the fix. This is the part that teaches judgement.

## How the pages are laid out

Three things repeat throughout, and recognising them will make the book faster to read.

FILE api/app/Models/Incident.php

```
// A grey bar like the one above always precedes code, and always
// names the exact file the code came from, relative to the project root.
```

A blue box explains *why* something is the way it is. The code shows what happens; these boxes explain the decision behind it.

**Careful.** An amber box is a mistake that is easy to make and unpleasant to diagnose. Read these even if you skim everything else.

**You should see:** A green box tells you what you should observe on your own machine if you followed along correctly. If you see something different, stop — do not continue until it matches.

**This broke.** A red box is a real failure that happened while building this system. Part V covers them in full, but they appear inline where the relevant code is introduced, so you meet the bug in the place it lives.

## The one rule

Type the commands. Do not copy and paste them, and do not read past a green box whose result you have not seen on your own screen.

The difference between reading about a system and being able to build one is almost entirely made up of the small failures you hit while typing — a wrong directory, a missing container, a typo in a hostname. Skipping them feels faster and teaches nothing.

# Part I

# Understanding the system

CHAPTER 1

# What IncidentFlow actually is

## 1.1 The problem

When something breaks in production — a payment service starts failing, a database runs out of disk — several things must happen at once. Someone has to be told. Someone has to take charge. Everyone else needs to see what is being done without interrupting the person doing it. And afterwards, someone has to be able to reconstruct exactly what happened and when.

That last part is the hard one. Chat threads get edited. Memories disagree. The engineer who fixed it at 3am writes the summary a week later.

IncidentFlow's single organising idea is that **the timeline is append-only**. Nothing written can be edited or deleted — not by an administrator, not by the person who wrote it, not by the API. If the record says the incident was acknowledged at 02:14, that is what happened. Almost every design decision in this book follows from that one commitment.

## 1.2 Who uses it

Five roles, deliberately few. Each can do strictly more than the one above it.

| Role | What it is for |
|---|---|
| `viewer` | Can read everything, change nothing. Support staff, managers, anyone who needs to know without being involved. |
| `reporter` | Can raise an incident, and comment. Cannot drive one. |
| `responder` | Works incidents: acknowledges, mitigates, resolves. |
| `incident_commander` | Owns an incident. Assigns people, changes severity, closes it out. |
| `administrator` | Manages the organisation: members, services, and the audit log. |

You will meet this table again in Chapter 4, where the same list appears as *screenshots of one page seen by different people*. That is the clearest way to understand a permission system: not as a table, but as buttons that are missing.

## 1.3 The shape of an incident

Every incident has a **severity** — how bad it is — and a **status** — where it is in its life.

### 1.3.1 Severity: how bad is it

| Value | Name | Meaning |
|---|---|---|
| `sev1` | Critical | Everything is down, or money is being lost right now. Wakes people up. |
| `sev2` | High | A major feature is broken for many users. |
| `sev3` | Moderate | Degraded, or broken for a few. |
| `sev4` | Low | Cosmetic or minor; fix it in normal hours. |

Severity is not decoration. `sev1` and `sev2` send email to every responder in the organisation; `sev3` and `sev4` do not. One enum value decides whether a hundred phones buzz at 3am — which is why changing severity is a permission a reporter does not have.

### 1.3.2 Status: where is it in its life

| Value | Name | Meaning |
|---|---|---|
| `open` | Open | Reported. Nobody has picked it up yet. |
| `acknowledged` | Acknowledged | Someone is actively working on it. |
| `mitigated` | Mitigated | Users are no longer affected, but the underlying cause is still there. |
| `resolved` | Resolved | Actually fixed. |
| `closed` | Closed | Reviewed and finished. Terminal. |

### 1.3.3 The transitions are a rule, not a suggestion

You cannot move an incident from any status to any other. The permitted moves are written down in exactly one place, and the API refuses everything else.

FILE api/app/Enums/IncidentStatus.php

```
public function allowedTransitions(): array
{
    return match ($this) {
        // A trivial incident can go straight to resolved without ceremony.
        self::Open         => [self::Acknowledged, self::Resolved],
        self::Acknowledged => [self::Mitigated, self::Resolved],
        // Mitigation that does not hold sends the incident back to active work.
        self::Mitigated    => [self::Resolved, self::Acknowledged],
        // Resolved is reversible: an incident that recurs was never resolved.
        self::Resolved     => [self::Closed, self::Acknowledged],
        // Closed is terminal. Post-closure findings belong in the postmortem.
        self::Closed       => [],
    };
}
```

Read that carefully, because three of those five lines encode a real opinion about how incidents work.

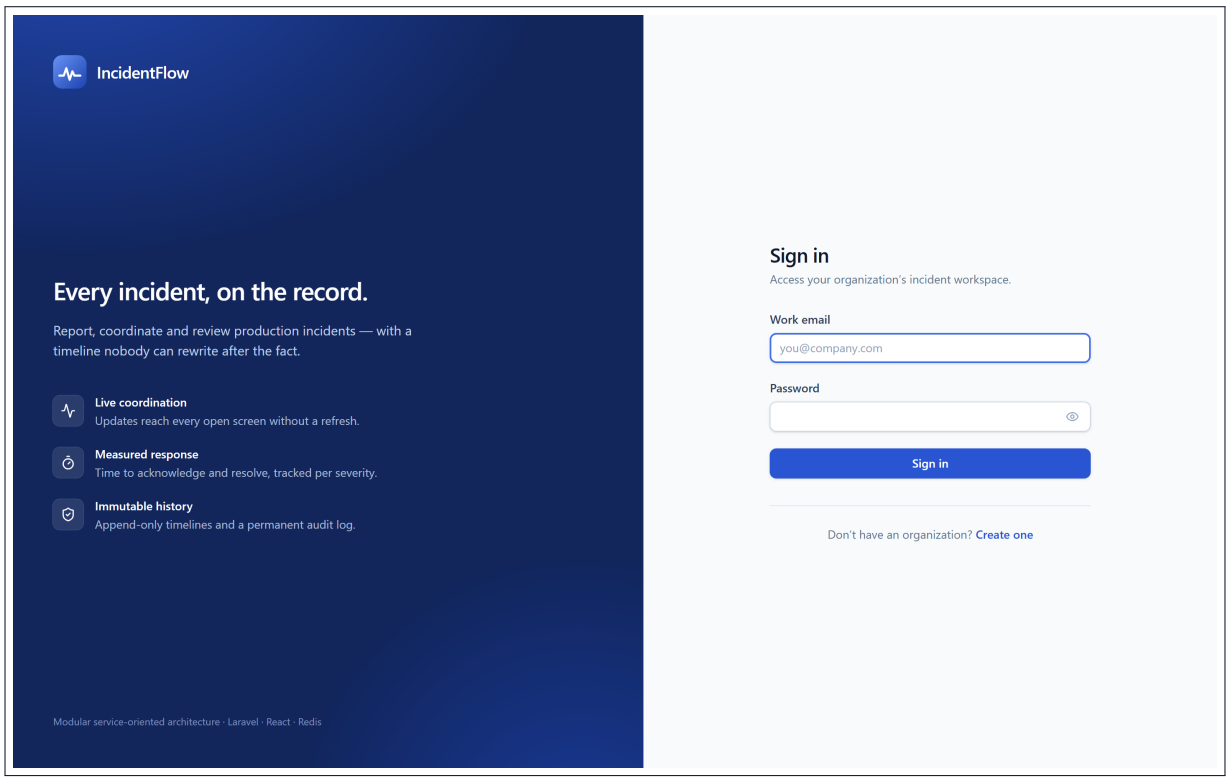
- **Open can jump straight to Resolved.** Forcing every trivial incident through an acknowledgement step produces a system people work around.
- **Mitigated can go *back* to Acknowledged.** A workaround that stops holding is not progress, and the timeline should say so.

- **Closed goes nowhere.** Once closed, the record is finished. Anything learned later belongs in the postmortem, which is a separate document — not a quiet edit to history.

**Careful.** This rule lives in the *enum*, not in the controller and not in the React app. The frontend also hides buttons for moves that are not allowed — but that is a courtesy to the user, not a security control. Anyone can send any HTTP request they like. If the rule lived only in the browser, it would not be a rule at all. You will see this pattern throughout: **the UI prevents mistakes, the API prevents abuse**, and only one of them is trusted.

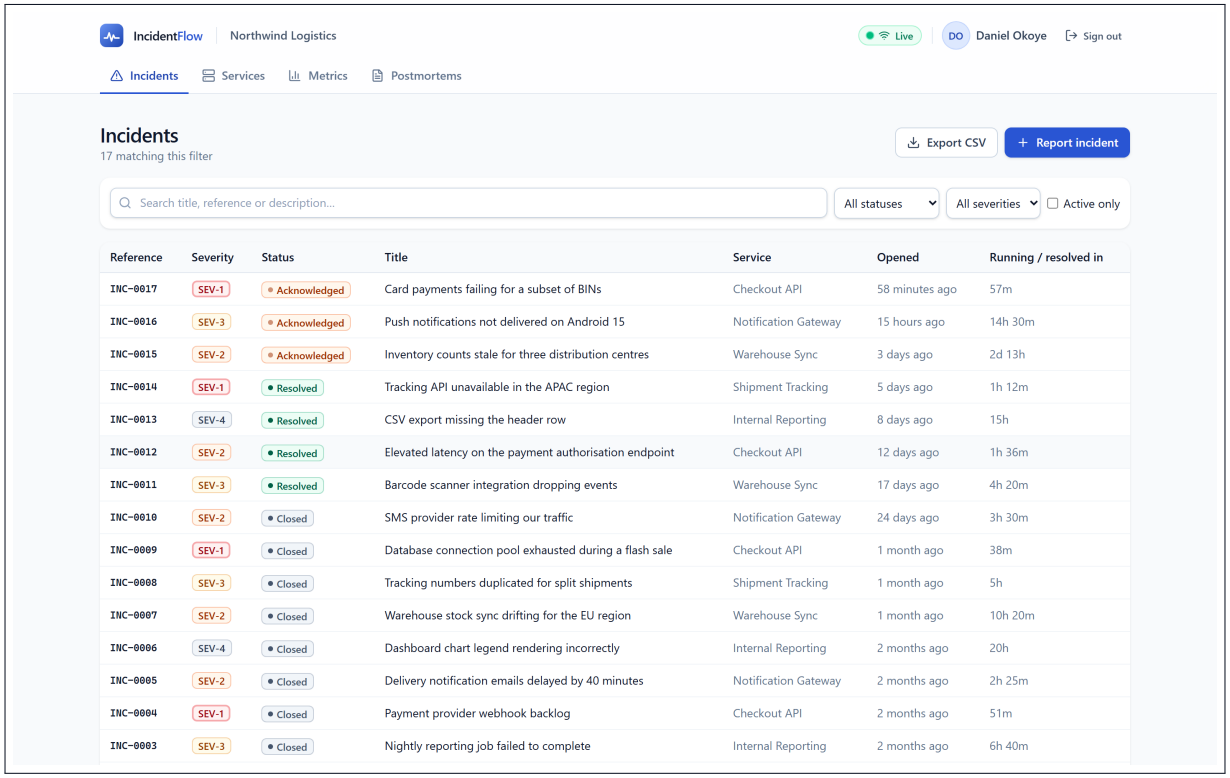
## 1.4 Your first look

Here is the sign-in screen — the whole system's front door.




The sign-in screen at `http://localhost:8080`. Chapter 5 takes this page apart line by line.

And here is what a signed-in incident commander sees: everything currently happening.




The incidents list, signed in as Daniel Okoye (incident commander). Seventeen incidents, seeded so the system has a realistic history to show.

Do not worry about details yet — Part II gives each screen a full chapter. For now notice only this: every coloured badge on that page is one of the enum values from the tables above. The pictures and the code are the same thing.

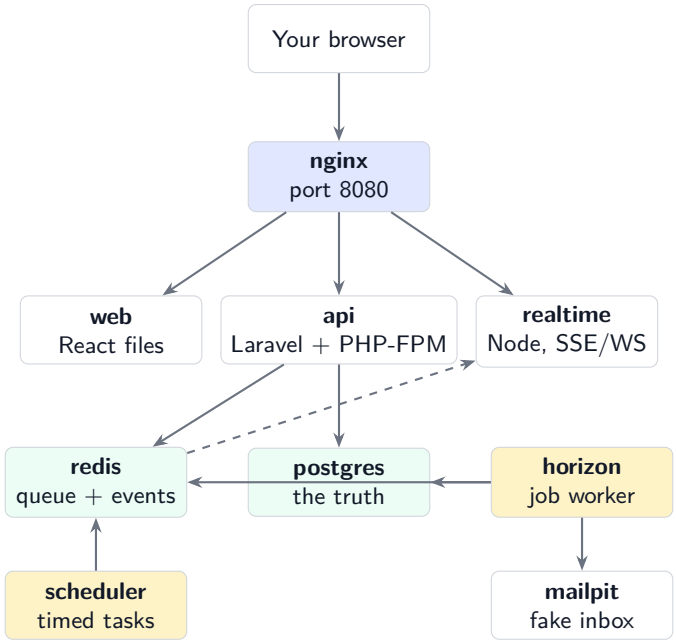
CHAPTER 2

# The architecture

## 2.1 Nine programs, not one

When you run IncidentFlow, nine separate programs start. That number surprises people. A student project is usually one program; this is nine, and every one of them earns its place.

Here is the whole system on one page. Read the arrows as “talks to”.




The nine containers. Solid arrows are direct connections; the dashed arrow is events flowing from Laravel to the realtime tier through Redis. Only nginx is reachable from outside.

## 2.2 What each one is for

| Container | Built from | Its job |
|---|---|---|
| `nginx` | nginx 1.27 | The front door. Everything from your browser arrives here and is routed onward. Nothing else is exposed. |
| `web` | React + Vite | Serves the compiled JavaScript and CSS. It has no logic — it asks the API for everything. |
| `api` | PHP 8.3 + Laravel | All the rules. Who may do what, what a valid transition is, what gets written. |
| `realtime` | Node.js 22 | Holds long-lived connections open so screens update without a refresh. |
| `postgres` | PostgreSQL 16 | Stores everything. The single source of truth. |
| `redis` | Redis 7 | Two jobs: carries live events from Laravel to realtime, and holds the queue of pending work. |
| `horizon` | same as api | Runs queued jobs — chiefly sending email — so the web request does not wait for them. |
| `scheduler` | same as api | Runs timed tasks, like sweeping notifications that were never delivered. |
| `mailpit` | mailpit | A fake inbox for development, so test email never reaches a real person. |

Notice that `api`, `horizon` and `scheduler` are built from *the same image*. One Dockerfile, three containers, three different start commands. Without Docker you would install PHP once and run three processes; here each gets its own isolated process, so a memory leak in the queue worker cannot take the API down with it.

## 2.3 Following one request all the way through

Abstract diagrams are easy to nod at and hard to remember. So follow one real action: **a responder clicks "Acknowledge" on an incident.**

1. The browser sends `POST /api/v1/incidents/42/status` to `localhost:8080`. That is **nginx**.
2. nginx sees the path starts with `/api/`, so it hands the request to **api** over FastCGI — not HTTP. (FastCGI is the protocol web servers use to talk to PHP.)
3. Laravel checks the JWT in the `Authorization` header: who is this?
4. It checks the *policy*: may a responder change this incident's status?
5. It checks the *enum*: is `open` to `acknowledged` a legal move?
6. Inside one database transaction it writes the new status to **postgres**, appends a timeline event, and creates notification rows.
7. The transaction commits. Only then does it push jobs onto **redis** and publish an event.
8. **horizon** picks the job off Redis and sends email through **mailpit**.
9. **realtime** is subscribed to Redis. It receives the event and pushes it down every open connection.
10. Every other open browser updates. Nobody pressed refresh.

**Careful.** Step 7 says "the transaction commits, *only then*" for a reason. If the job were queued before the commit, Horizon — a separate process — could pick it up and look for an incident that has not been written yet. It would find nothing and fail. This is one of the classic distributed-system bugs, and the ordering in that step is the entire defence against it.

## 2.4 Why not one program?

You could build all of this as a single Laravel application. Many people would. Three things here genuinely cannot be done that way.

**Long-lived connections**
: PHP-FPM assigns a worker process per request. A connection held open for an hour would occupy a worker for an hour. Twenty users watching a dashboard would consume the whole pool and the API would stop answering anything at all. Node handles thousands of idle connections on one process, because that is what an event loop is good at. Hence a separate `realtime` service.

**Slow work**
: Sending an email takes as long as the mail server takes. If that happened inside the request, the responder's browser would spin while SMTP negotiated. So the API writes a row, returns immediately, and `horizon` does the sending afterwards.

**Timed work**
: Something must run every minute regardless of whether anyone is using the system. A web request cannot do that, because there might not be one. Hence `scheduler`.

This is a *modular service-oriented* architecture, not microservices. The distinction matters: there is one database and one deployable unit. The services are split along the lines where the *runtime model* differs — request/response, long-lived connection, background work — not along business domains. Splitting by domain is what produces microservices, and with them distributed transactions, which this system deliberately does not have.

## 2.5 The security boundary

One rule, and it explains most of the configuration you will read later: **only nginx is reachable from outside**.

| Service | Reachable from your machine? | How |
|---|---|---|
| nginx | yes | `localhost:8080` |
| mailpit | yes (development only) | `localhost:8025` |
| postgres | development only | `localhost:5432` |
| redis | development only | `localhost:6379` |
| api | no | only nginx, over the internal network |
| realtime | no | only nginx |
| web | no | only nginx |
| horizon | no | has no network port at all |
| scheduler | no | has no network port at all |

In production those "development only" rows are closed too. You will see exactly how in Part III, in `docker-compose.prod.yml`.

CHAPTER 3

# Every file, and why

Placeholder.

CHAPTER 4

# Roles

Placeholder.

CHAPTER 5

# Sign in

Placeholder.